

AquaGrid

How every part of the system works, what each file is responsible for, and the things you must not “fix”.

What this document is

Read section 01 and 02. Read the rest when you touch that file.

AquaGrid decides, hour by hour, whether to run an island's desalination pump. It does that using a trained demand forecaster, a live weather forecast, a battery, the rain, and a set of rules that trade diesel against the risk of running the tank dry. It then explains every decision in plain language and watches for four kinds of failure.

The whole thing runs **on the phone with no server**. That is the single most important architectural fact and section 02 explains why.

01	Run it in five minutes	setup, three commands
02	Architecture, and why there is no backend	read this before changing anything
03	Repository map	every file, one line each
04	The data contract — forecast.json	the only interface between Python and the app
05	The ML pipeline	generate_data.py, train_model.py
06	The scheduler	eight rules and the physics they share
07	Baselines	what we compare against, and why it is fair
08	The four detectors	leak, soiling, fouling, fuel
09	Advisory — talking to the village	free-solar windows, WHO minimum
10	Any location	shipping the model, not its predictions
11	The LLM layer	Groq, and the fallback that always works
12	The screen, card by card	what each component shows
13	Testing and definition of done	one command
14	Things you must not “fix”	tuned values that look wrong
15	Demo script	90 seconds, in order

Run it in five minutes

Two halves: Python trains, the app consumes.

Train the model and export the forecast

```
cd ml
python -m venv ../.venv
../.venv/bin/pip install -r requirements.txt
../.venv/bin/python generate_data.py    # 17,520 rows of synthetic history
../.venv/bin/python train_model.py      # trains, evaluates, writes ../app/assets/forecast.json
```

`train_model.py` prints MAE, R^2 and feature importances, fetches a live 48-hour forecast from Open-Meteo, and writes the JSON the app ships with. If the network is blocked it falls back to a modelled clear-sky curve and says so in `weather_source`. **The build never blocks on the network.**

Run the app

```
cd app
npm install
cp src/config.example.js src/config.js    # optional: paste a Groq key
npx expo start                            # add --ios for the simulator
```

Install **Expo Go** on a phone and scan the QR code.

Verify the logic without a phone

```
cd app && node src/logic/test.js
```

Prints all three strategies, the savings, the leak result and the lookup-table fidelity. Exits non-zero if any check fails. Everything in `src/logic/` is free of React imports specifically so this works.

IF THE PHONE CONNECTS BUT NEVER LOADS

Campus and guest wifi block device-to-laptop traffic. Use `npx expo start --tunnel`.

AFTER CHANGING A FUNCTION SIGNATURE IN LOGIC/

Fast Refresh keeps stale module closures and you will get errors like `Cannot create property 'sockWh' on number` even though the code is correct. Restart with `npx expo start --clear`. This already bit us once.

NEVER

Do not run `npx expo prebuild` or `eas build`. Expo Go only. There is no time and no need.

Architecture, and why there is no backend

Read this before you propose adding an API.



Python trains and evaluates the model on a laptop and exports everything the app needs into one JSON file. The app bundles that file as a static asset. Scheduling, rainwater harvesting, battery dispatch, leak and asset detection, and all charting run in JavaScript on the device.

Why this and not a server

A laptop-hosted API reached from a phone means LAN addresses, ngrok tunnels, CORS and cold starts. Those are the four most common ways a demo dies. Removing the server removes all four.

More importantly it is the *correct* design for the problem. An island whose uplink is a satellite dish cannot depend on a cloud round-trip to decide whether to run its pump. **The app works in airplane mode.** Say that out loud in the demo.

IF A JUDGE ASKS "IS THE AI REAL?"

Yes. The model is trained and evaluated in Python on a chronological split, the metrics are in the README, and its predictions drive every decision. "Trained offline, inference on-device" is how most production mobile ML actually ships. Since we also export the model as a lookup table (section 10), the forecaster genuinely runs on the phone for locations the laptop never saw.

What is and is not synchronous

All scheduling is **synchronous** and lives in `useMemo`. Only the two LLM calls are async and they live in `useEffect`. Never mix them — putting a promise in `useMemo` is the classic way to break this app.

Repository map

Every file that matters, one line each.

ml/ – the Python half

<code>generate_data.py</code>	Synthesises two years of hourly demand with real, readable structure. Auditable in one screen.
<code>train_model.py</code>	Trains the forest, evaluates on a chronological split, fetches weather, exports <code>forecast.json</code> .
<code>requirements.txt</code>	numpy, pandas, scikit-learn. Nothing else.

app/src/logic/ – pure JavaScript, no React, node-testable

<code>scheduler.js</code>	The eight rules, the shared physics in <code>settleHour</code> , and the warm-up replay.
<code>baselines.js</code>	Fixed-timer and reactive strategies. Same physics, different rule.
<code>leak.js</code>	Burst-pipe detection by residual analysis, plus the simulated tank sensor.
<code>assets.js</code>	Panel soiling and RO membrane fouling.
<code>fuel.js</code>	Diesel inventory against barge lead time.
<code>advisory.js</code>	Free-solar windows and litres per person per day.
<code>demand.js</code>	The exported model, running on-device.
<code>plant.js</code>	Scales a plant to an arbitrary population.
<code>test.js</code>	The definition-of-done harness. <code>node src/logic/test.js</code> .

app/src/lib/

<code>geo.js</code>	Open-Meteo geocoding and live weather for any island.
<code>ai.js</code>	Groq briefing, question answering and public notice, each with a deterministic fallback.

app/src/components/ – 16 components

Covered card by card in section 12.

Other

<code>app/assets/forecast.json</code>	The single interface between the two halves. Generated, not hand-edited.
<code>app/src/config.js</code>	Gitignored. Holds the Groq key. Copy from <code>config.example.js</code> .
<code>docs/index.html</code>	The project website. Charts generated from the real forecast.

The data contract — forecast.json

Everything the app renders comes from here. Import it directly; Metro bundles JSON with no config.

```
import forecast from './assets/forecast.json';
```

KEY	WHAT IT IS
island	Name, lat, lon, population.
weather_source	"open-meteo (live)" or the offline fallback label. Drives the header pill.
plant	Tank, pump, output, diesel rate, reserve floor, target, array, catchment, runoff, battery, fuel stock, barge interval, storm threshold.
model	MAE, RMSE, R ² , naive baseline, row counts, feature importances.
demand_model	The 15,120-cell lookup table. Section 10.
maintenance	90 days of RO specific-energy readings, plus design and clean threshold.
validation_curve	72 held-out hours, actual vs predicted, for the model detail modal.
warmup	The 24 hours <i>before</i> the horizon. Replayed to derive the starting tank level.
hourly	48 entries: time, hour, solar_kw, temp_c, cloud_pct, rain_mm, wind_kmh, predicted_demand_lph.

WHY WARMUP EXISTS

Without it every island opens at exactly the seeded percentage — 59% — and the biggest number on screen looks like a constant rather than a measurement. Replaying yesterday’s real weather makes hour zero a consequence. Funafuti opens at 32%, Reykjavík at 41%.

The ML pipeline

ml/generate_data.py and ml/train_model.py

generate_data.py

Two years of hourly demand with structure a model can genuinely learn, not noise it can only memorise:

- Twin daily peaks — morning wash 06:00–08:00, evening cooking 18:00–20:00
- Weekends +30% — people are home, laundry day
- Tourist season (Jun–Aug) +22%
- Demand rises ~3.5% per °C
- Gaussian noise so nothing is perfectly predictable

Anyone can read this file and verify the patterns are real, which is exactly what you want a judge to be able to do.

train_model.py

RandomForestRegressor(n_estimators=180, max_depth=14, min_samples_leaf=3) on a **chronological split** — the last 60 days are held out, so the model is tested on the future, never on shuffled rows. This is the single most defensible thing about the model and you should say “chronological split” out loud.

METRIC	VALUE
MAE	45.3 L/h
RMSE	57.0 L/h
R ²	0.987
Naive “predict the mean”	409.7 L/h
Improvement over naive	89%

Feature importances: hour 0.897, is_tourist_season 0.032, is_weekend 0.031, day_of_week 0.028, temp_c 0.012. Hour dominating is correct — that is what a village water profile looks like.

The script then fetches Open-Meteo with `past_days=1&forecast_days=3&timezone=auto`, converts shortwave radiation to kW through the panel area, predicts demand forward, and writes the JSON.

DO NOT CHANGE TIMEZONE=AUTO

Funafuti is UTC+12. With UTC timestamps, peak solar lands at local midnight and every pump decision inverts. This was a real bug we already fixed.

The scheduler

app/src/logic/scheduler.js – the core of the system

Shared physics: settleHour()

Every strategy — ours and both baselines — applies its decision through the same function, so comparisons differ only in the rule. Order of operations per hour:

1. **Solar first.** `fromSolar = min(solar_kw, pumpKw)`
2. **Battery next.** Discharge before burning anything, capped at the pump draw.
3. **Diesel last.** `dieselL = remaining_kw × diesel_l_per_kwh`
4. **Charge.** Surplus solar charges the battery at round-trip efficiency; what will not fit is *curtailed* and counted.
5. **Water.** `tank + pumped + harvested - demand`, clamped to capacity. Overflow is counted as spill.
6. **Fuel stock decrements.** Shortage hours increment if the tank hits zero.

`harvest = rain_mm × roof_catchment_m2 × runoff_coeff`. One millimetre over one square metre is one litre.

The eight rules — strict priority, first match wins

#	RULE	FIRES WHEN
0	Storm prep	Cyclone-force wind within 24 h. Fill to 100% regardless of cost.
1	Emergency	Below the 25% reserve floor.
2	Rain hold	More rain than demand in the next 6 h, reserves comfortable. Hold; the tank fills free.
3	Free solar	Sun alone clears the full pump draw and the tank is below target.
4	Stored solar	Shortfall ahead and the battery can carry the pump.
5	Pre-emptive hybrid	Shortfall ahead with partial sun. Spend a little diesel now to avoid a lot tonight.
6	Reluctant diesel	Shortfall ahead, no sun, empty battery, reserves thin.
7	Hold	Everything else. Wait for the sun.

The lookahead

```

horizon      = next 12 hours
projDemand   = sum of predicted demand
projSolarHours = hours where solar_kw >= pumpKw
projRainL    = sum of rain × catchment
willBreach   = tank + (projSolarHours × outLph) + projRainL - projDemand < floorL

```

Rules 2, 4 and 5 are the intelligence of the system. They are the only ones acting on information the plant does not yet have. If none of them fire in a demo run, the plant parameters have drifted — see section 14.

warmUpTank()

Runs the scheduler over the `warmup` array and returns the final tank level, which becomes `initial_tank_l` for the displayed horizon.

Baselines

`app/src/logic/baselines.js` – what makes the savings number honest

Both run over the **identical forecast with identical physics** — same rain, same battery, same tank, same `settleHour`. Only the decision rule differs. A judge will ask how the savings were computed; this is the answer.

fixedTimerSchedule

Pumps 01:00–05:00 every day regardless of sun, rain or tank level. This is how these plants are *actually* run today — a wall timer set to mainland off-peak grid hours. On a solar island it is exactly backwards, because those hours have no sun.

reactiveSchedule

Pumps whenever the tank drops below 80%. Responds to the tank but is blind to the forecast, so it pumps at night as readily as at noon and makes water it is about to spill.

STRATEGY	DIESEL	COST	CO ₂	PUMP H
AquaGrid	15.3 L	\$25	41 kg	6
Fixed timer	70.0 L	\$112	188 kg	8
Reactive	79.2 L	\$127	212 kg	11

78% less diesel than the fixed timer. Note it delivers comparable water in *fewer* pump hours — it chooses which hours.

The four detectors

One idea: compare what the plant does against what the forecast says it should.

Burst pipe — leak.js

```
expectedDrop = demand - pumped - harvested + spilled
actualDrop   = sensor[i-1] - sensor[i]
residual      = actualDrop - expectedDrop    // positive = unexplained loss
sigma         = std(residual over first 12 h) // known-good baseline
ALERT when residual/sigma > 3 for 3 consecutive hours
```

A 450 L/h burst injected at hour 14 is caught at **hour 16**, estimated at 452 L/h with 99% confidence.

DO NOT DROP HARVESTED FROM EXPECTEDDROP

Rainwater is free inflow the forecast knows about. Omit it and every shower reads as a negative leak. This broke once already.

Dirty panels — assets.js / detectSoiling

Compares metered array output against the irradiance-derived prediction, over daylight hours only — at night both are zero and the ratio is meaningless. Reports shortfall percentage and kWh not collected. Currently: **13% below predicted, 90 kWh lost.**

Fouling membranes — assets.js / projectFouling

Specific energy (kWh per m³) is the standard health metric for a reverse-osmosis train and rises as membranes foul, because the pump pushes harder for the same flow. Ordinary least squares on 90 days of daily readings, projected to the clean-in-place threshold. Currently: **+11.1% from design, threshold in 40 days.**

Fuel exhaustion — fuel.js

Burning too much diesel and running out of diesel are different failures.

STRATEGY	BURN	LASTS	VS 12-DAY BARGE
AquaGrid	8 L/day	39 days	holds
Fixed timer	35 L/day	8.6 days	dry 3.4 d early
Reactive	40 L/day	7.6 days	dry 4.4 d early

This is arguably a stronger claim than the percentage. The barge does not come early because you ran out.

Advisory — talking to the village

`app/src/logic/advisory.js`

Everything else schedules supply. This is the one piece that addresses demand. The cheapest litre of diesel is the one nobody needed.

`freeSolarWindows()`

Runs of consecutive hours where the pump is already running on sun or stored sun — hours when extra village demand costs the island nothing. Turned into chips in the UI and a sentence in the notice.

`rationingOutlook()`

Supply per person per day against the **WHO minimum of 15 L** for drinking, cooking and basic hygiene. Currently 18.2 L — 1.2× the floor, so no notice is warranted. When it falls below, the LLM drafts the public notice instead of the “when water is cheap” one.

The advisory has its own system prompt written for *residents*: no kilowatts, no plant jargon, say what to do and when.

Any location

`app/src/logic/demand.js`, `plant.js` and `lib/geo.js`

We ship the model, not only its predictions

Every feature the forecaster uses is categorical — hour, day of week, weekend, tourist season — except temperature, a single continuous variable. So the whole function can be evaluated exhaustively on a grid and shipped as a table.

15,120 cells: 2 seasons × 7 days × 24 hours × 45 temperature buckets at 0.5 °C. Shape is `table[tourist][dow][hour][tempBucket]`, in L/h for the reference population.

This is the model, not an approximation. The only loss is temperature discretisation, costing a **mean 5.5 L/h** — 12% of the model's own 45.3 L/h error. `test.js` asserts it stays below a quarter of the model MAE. `demand.js` interpolates between adjacent buckets because a forest is a step function in temperature.

plant.js — sizing for an arbitrary island

Tank, pump, output, panel area, roof catchment, battery and fuel storage all scale linearly with population off the reference plant. That holds the ratios that drive decisions constant — specific energy per m³, days of storage, and how many hours a day solar clears the pump draw — so **only the weather genuinely differs between locations**. State this as an assumption; it is one.

geo.js

Open-Meteo geocoding, then the same forecast endpoint with `past_days=1&timezone=auto`. Both free, no key.

SEARCH QUIRK, ALREADY HANDLED

The geocoder matches settlement names only, so "Malé Maldives" and "Apia, Samoa" return nothing on a literal query. We retry on the first token and prefer results whose country or region matches the rest.

ISLAND	POPULATION	SAVEDWHY
Apia, Samoa	40,407	80%abundant sun
Malé, Maldives	103,693	70%equatorial, steady
Santorini, Greece	15,453	64%strong but seasonal
Reykjavík, Iceland	118,918	23%little sun to schedule

Reykjavík at 23% is the honest answer, not a bug. Where there is little sun there is little for a scheduler to win.

The LLM layer

app/src/lib/ai.js

Three functions, all with the same two-path shape:

<code>getBriefing</code>	The operator's shift briefing.
<code>askOperator</code>	Answers the three preset questions.
<code>getAdvisory</code>	The public water notice, in a different voice.

The template is the primary path — deterministic, instant, works in airplane mode. The Groq call is an enhancement raced against a 6-second timeout. Any error, timeout or missing key falls silently back, and the UI labels which one wrote the text so we never claim an LLM wrote a template.

Provider

Groq free tier, no credit card. Model is `openai/gpt-oss-120b` — **open-weight**, so no proprietary model is in the loop. `groq/compound-mini` is tried as a second model before the template.

GPT-OSS IS A REASONING MODEL

Without `reasoning_effort: 'low'` it spends the entire token budget thinking and returns *empty content*. We hit exactly this. Also note `llama-3.3-70b-versatile` is decommissioned on Groq — do not put it back.

KEY HANDLING

`src/config.js` is gitignored and no key is committed. The key lives client-side because there is no backend to hide it behind — the deliberate trade for an offline app. Say so in the README rather than hoping nobody notices. Beyond a demo this belongs behind a proxy, and the demo key should be revoked afterwards.

The screen, card by card

One SafeAreaView, one ScrollView, no navigation library.

COMPONENT	WHAT IT SHOWS
Header	Island, population, live/offline pill from <code>weather_source</code> , timestamp.
LocationPicker	Current island and a search sheet. Anchored to the top so the keyboard cannot cover the results.
StormBanner	Only when a cyclone is in the outlook. Explains why fuel rules are overridden.
TankGauge	Tank %, litres, hours of supply, tick marks at the 25% floor and 92% target.
DecisionCard	The most important element. Current action colour-coded by source, with the scheduler's own reason string, demand, production, rain inflow, diesel and battery draw.
BriefingCard	The operator briefing, badged <code>AI</code> or <code>on-device</code> .
AskPanel	Three preset questions. Presets not free text, so nobody types on camera.
SolarDemandChart	Solar area, demand line, rainfall bars, dashed rule at the pump draw. That rule makes the whole strategy legible.
PumpTimeline	24 bars coloured by source. Tap one to read that hour's reason.
BatteryCard	State of charge over 24 h, plus solar-to-pump, from-battery and curtailed kWh.
WaterSourcesCard	Rain vs desalinated split. On Tuvalu this is the headline.
SavingsCard	Three strategies, diesel, cost, CO ₂ . Hero percentage computed, never hardcoded.
FuelCard	Litres remaining, days to empty, barge countdown, and the same for both baselines.
LeakPanel	Green all-clear or red alert, plus Simulate burst pipe .
AssetHealthCard	Soiling and fouling, with the 90-day regression chart and Simulate dirty panels .
AdvisoryCard	Public notice, free-solar window chips, litres per person against the WHO marker.
ModelCard	MAE, R ² , 89% vs naive, feature-importance bars, and a modal with predicted vs actual.
DemoBar	Simulate cyclone. Every switch changes a real input; nothing fakes an outcome.

Charts are hand-drawn with `react-native-svg` . **Do not add a charting library** — version drift against the Expo SDK is a real risk and each chart is about sixty lines.

Testing and definition of done

One command, exits non-zero on failure.

```
cd app && node src/logic/test.js
```

It asserts: zero shortage hours, a clean run produces no leak alert, an injected burst *is* caught within three hours, savings are positive, and the lookup table stays within a quarter of the model's MAE. It prints the full comparison table, the 24-hour timeline and the warm-up result.

Before recording the video

- `python train_model.py` runs clean and prints MAE / R^2 / importances
- `forecast.json` has 48 hourly entries and 24 warm-up entries
- `node src/logic/test.js` passes
- The app loads in Expo Go on a physical phone
- Savings are computed from both schedules, not hardcoded
- “Simulate burst pipe” alerts within 3 simulated hours
- Airplane mode: every screen renders; only the briefing degrades, and it degrades to the template rather than an error
- No API key committed
- README quotes the metrics `train_model.py` actually printed

EXPO GO DEV OVERLAY

The floating blue gear button is Expo Go's, not ours. Drag it into a corner before recording.

Things you must not “fix”

Values tuned so the system has a real decision to make. Every one of these looks wrong and is not.

VALUE	WHY IT IS WHAT IT IS
<code>timezone=auto</code>	Funafuti is UTC+12. UTC timestamps put peak solar at local midnight and invert every decision.
<code>array_m2 = 320</code>	Peak output only clears the 50 kW pump draw for two or three midday hours. Enlarge it and every hour becomes a solar hour, the reserve floor never binds, and rules 4–6 become dead code.
<code>desal_pump_kw = 50</code>	Raised from 42 for the same reason. At 42 the scheduler coasted on solar alone and reported an implausible 100% saving.
<code>tank_capacity_l = 60000</code>	About 2.7 days of demand. Deliberately tight, so the reserve floor actually binds.
<code>roof_catchment_m2 = 900</code>	Rain covers ~26% of demand. At 2,500 it covered 72% and the scheduler had nothing to do.
<code>diesel_stock_l = 300</code>	25% of the day tank with 12 days to the barge. This is the state where a good scheduler stops being about money.
Fouling drift rate	Tuned so the clean-in-place date lands ~6 weeks out — near enough to act on, far enough to be a prediction rather than an alarm.

THE GENERAL TRAP

Three separate times, a change made the plant *too capable* and the scheduler stopped having to decide anything — which reads to a judge as a broken demo, not a good one. If you change a plant parameter, re-run `test.js` and confirm **all five sources still fire**: solar, battery, hybrid, diesel and hold.

Also do not

- Add a backend, navigation library, or charting library.
- Put a promise in `useMemo`.
- Hardcode the savings number. A judge will ask how it was computed.
- Reinstate `llama-3.3-70b-versatile`, which no longer exists on Groq.

Demo script

90 seconds, shot on the phone, one take, phone in a stand.

TIME	BEAT
0:00–0:10	The problem. “On an isolated island, desalination is the biggest power draw, and it usually runs on a dumb timer at night, on 100% diesel.”
0:10–0:22	Open the app. Tank gauge, current decision, read the reason line out loud. It is a full sentence, not a status code.
0:22–0:38	Chart and timeline. Point at the dashed pump-draw line. Green hours are free solar, cyan is stored solar running the pump after dark, amber is a deliberate partial-solar top-up.
0:38–0:48	Rain. “Tuvalu runs on rainwater; desal is the backup. A quarter of this water fell out of the sky, and the scheduler holds the pump when a shower is coming.”
0:48–0:58	Savings, then fuel. Say 78% out loud, then scroll to the fuel card: “and both naive schedulers run out of diesel before the next barge.”
0:58–1:10	Tap Simulate burst pipe. The alert fires. “It caught that from the forecast residual, not a threshold — no human would have noticed for hours.”
1:10–1:20	Change location. Search another island. Everything recomputes on-device.
1:20–1:30	Airplane mode. Everything still works. “Built for an island with no reliable connectivity.”

ENDING

Ending on airplane mode is the strongest closing beat available. Do not cut it.